

# Guidelines to Prompt LLMs for Code Generation

Midolo et al. (2026) derived and evaluated development-specific prompt optimization guidelines using an iterative, test-driven approach to automatically refine code generation prompts.

## Key Contributions:

- **10 Guidelines** for prompt improvement: specifying I/O, pre/post conditions, providing examples, clarifying ambiguities.
- **Empirical Evaluation** with 50 practitioners rating usage and perceived usefulness.
- **LLM Coverage:** GPT-4o-mini, Llama 3.3 70B, Qwen2.5 72B, DeepSeek Coder V2.

**Relevance to Agentic AI:** These guidelines directly inform Steps 1–3 (Goal Definition, Task Decomposition, Planning) by providing evidence-based prompt patterns that improve LLM-driven agent reliability.

# Example: Applying the 10 Guidelines

**Task:** Create a Jupyter notebook that plots a sine wave at 10 Hz.

## Before

*"Make a notebook with a sin plot."*

**Missing:** frequency, sample rate, axis labels, libraries.

## After

*Create a Jupyter notebook plotting a sine wave at  $f = 10$  Hz.*

**Requirements:** Use `numpy` and `matplotlib.pyplot`.

**Algorithm:** Sample at  $f_s = 1000$  Hz over  $t \in [0, 1]$  s; compute  $y = \sin(2\pi ft)$ .

**I/O format:** Title "Sine Wave (10 Hz)"; x-axis "Time [s]", y-axis "Amplitude".

**Pre/Post:**  $f_s > 2f$  (Nyquist); output  $y$  matches  $t$  length.

**Errors:** Raise `ValueError` if  $f_s \leq 0$  or  $f \leq 0$ .

**Example:** For  $f = 1$ ,  $t = [0, 0.1, \dots, 0.9]$ , expect  $y[0] = 0$ ,  $y[2] \approx 0.588$ .

**Guidelines applied:** Requirements ✓ Pre/Post-conditions ✓ I/O Format ✓ Exceptions ✓ Algorithmic Details ✓ Variable Naming ✓ Clarity ✓ Examples ✓ Assertive Language ✓

# Midolo et al. (2026): 10 Prompt Guidelines

| Guideline           | Description                                                       | Applied |
|---------------------|-------------------------------------------------------------------|---------|
| Requirements        | Make required packages/libraries explicit; explain their purpose. | 19%     |
| Pre-conditions      | State conditions that must hold before execution.                 | 7%      |
| Post-conditions     | Specify expected guarantees after execution.                      | 23%     |
| I/O format          | Clarify input/output types, shapes, and special cases.            | 44%     |
| Exceptions          | Explicitly define exceptions to raise and error messages.         | 12%     |
| Algorithmic Details | Detail formulas, definitions, and complex logic.                  | 57%     |
| Variable Mentioning | Use consistent terminology for variables.                         | 3%      |
| Unclear Conditions  | Avoid “otherwise”; describe both branches explicitly.             | 1%      |
| More Examples       | Add doctests/run examples.                                        | 24%     |
| Assertive Language  | Use “must” instead of “should”; avoid ambiguity.                  | 9%      |

**Takeaway:** Algorithmic details and I/O format are the most frequently needed refinements; linguistic fixes (variable naming, unclear conditions) are rarer but still valuable.

# CodePromptEval: Dataset & Methodology

Khojah et al. (2025) introduced **CodePromptEval**, a full factorial dataset to evaluate how prompt techniques—and their interactions—impact function-level code generation.

## Experimental Design

- **221 tasks** from CoderEval  $\times$  **32 variations** = **7,072 prompts**
- **5 techniques:** few-shot, CoT, persona, signature, packages
- **3 LLMs:** GPT-4o, Llama3-70B, Mistral-22B (3 runs each, temp=0.2)

**Relevance:** Systematic benchmark for evaluating prompt engineering strategies in code generation agents.

## Evaluation

- **Correctness:** Pass@1
- **Similarity:** CrystalBLEU
- **Quality:** code smells + complexity

# Key Findings: The Correctness–Quality Trade-off

## Correctness (RQ2.1)

- **Signature + few-shot** = best Pass@1 (57.5% for GPT-4o)
- Best vs. worst combo: only **10 pp gap**
- Combining techniques often **cancels out** benefits

## Quality (RQ2.3)

- CoT, persona, packages → **fewer code smells**
- Few-shot + signature → **more code smells** (71% lack docstrings)
- Llama3 produces simpler code than GPT-4o

## Critical Trade-off (L3)

Correctness techniques (signature, few-shot) worsen quality.

Quality techniques (persona, CoT) reduce pass rates.

**Agents must optimize for the right metric per task.**

**Relevance to Agentic AI:** Blindly stacking prompt techniques is counterproductive.

# How Practitioners Actually Use GenAI for SE

Otten et al. (2025) surveyed **72 software practitioners** across 6 SE tasks to characterize real-world GenAI usage, prompting strategies, and reliability perceptions.

## Tool Adoption

- **ChatGPT** dominates (83.1%), Copilot second (43.7%)
- Web + IDE equally preferred (63.9% vs. 62.5%)
- Daily use correlates with task coverage ( $\rho = 0.325$ )

## Workflow Impact

- 77.5% report GenAI changed their approach
- Replaced web searches and boilerplate generation
- Shifted focus to higher-level design

**Relevance:** Understanding practitioner workflows informs how agents should integrate into existing development practices.

# Prompting in Practice: Strategies & Reliability

## Conversation Strategies

- **Iterative multi-turn** preferred over single-shot
- Incremental refinement + feedback loop = most common
- Proficient users require **10+ exchanges** for complex tasks

## Awareness vs. Adoption

- 74–94% familiar, but <50% regularly use them
- Intermediates (6–10 yr) = most active adopters

## Reliability Perception

- **Most reliable:** documentation (avg. 3.06/4)
- **Least reliable:** debugging, complex algorithms (avg. 2.34–2.38/4)
- #1 issue: “bug fixes miss root causes”

**Relevance to Agentic AI:** Agents should emulate iterative refinement, embed techniques into interfaces, and prioritize reliability for debugging.

# References I

- [1] Ranim Khojah et al. “The Impact of Prompt Programming on Function-Level Code Generation”. In: *arXiv preprint arXiv:2412.20545* (2025). URL: <https://arxiv.org/abs/2412.20545>.
- [2] Alessandro Midolo et al. “Guidelines to Prompt Large Language Models for Code Generation: An Empirical Characterization”. In: *arXiv preprint arXiv:2601.13118* (2026). URL: <https://arxiv.org/abs/2601.13118>.
- [3] Daniel Otten et al. “Prompting in Practice: Investigating Software Practitioners’ Use of Generative AI Tools”. In: *arXiv preprint arXiv:2510.06000v2* (2025). URL: <https://arxiv.org/abs/2510.06000v2>.